Edge cases (a template with the wrong variables) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `prompt_template_practice.py`: a broken template can fail in two very different places — while building the prompt, or only once the API call goes out — and knowing which one changes how fast you can debug it. Seeing both a missing and an extra variable here, once, on purpose, means you recognize the failure instantly later. **If not:** the Build Task's `prompts.py` requirement to "fail the same clear way" would ask you to guarantee behavior you'd never actually observed happening.

Solution

```
# prompt_template_practice.py
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_template("Answer: {question}")

# A missing variable fails right away, before any API call. Catch KeyError
# specifically, so a genuinely different bug would still surface.
try:
    prompt.invoke({})
except KeyError:
    print("Missing variable -> KeyError, before any API call")

# An extra, unused key is silently ignored — not an error. (Easy to assume
# the opposite, so it's worth seeing once.) The filled prompt is a list of
# messages; [0] is the only one here.
result = prompt.invoke({"question": "What is LCEL?", "extra": "unused"})
print(f"Extra variable -> no error: {result.messages[0].content}")
```

Expected output:

```
Missing variable -> KeyError, before any API call
Extra variable -> no error: Answer: What is LCEL?
```

Which one should you write? Intermediate's specific-`KeyError` catch is what this document's Build Task needs — a clear error while building the prompt, not silently left blank. A stricter check — reading `prompt.input_variables` directly and validating both missing *and* extra (typo'd) keys before ever calling `.invoke()` — is worth adding anywhere you're building prompts from user input or config, where a typo'd key is a real, easy mistake; not required for this exercise or the Build Task.